TECHNICAL DESIGN DOCUMENT

AskBeforeAnswer — System & Pipeline Architecture

Document ID	TDD-ASKBEFOREANSWER-001
Version	1.0.0
Status	**Draft — For Review**
Date	September 2026
Owner	ML Engineering & MLOps
Audience	ML Engineering, MLOps, Applied Research, Platform

Classification: INTERNAL / ENGINEERING

1. Purpose & Scope

This Technical Design Document (TDD) describes **how** AskBeforeAnswer is built: the repository and module architecture, the Qwen 2.5 7B / Unsloth model design, the SFT → DPO/GRPO training pipeline, configuration and versioning tooling, experiment tracking and publication integrations, CI/CD, and the production deployment architecture. It complements TRD-ASKBEFOREANSWER-001, which defines *what* the system must satisfy; this document defines the concrete engineering design that satisfies it.

The scope of this document covers:

- Repository layout, Python package structure, and separation-of-responsibility design
- Model architecture and hardware acceleration stack (Unsloth, Flash Attention, xFormers)
- The SFT → DPO/GRPO training pipeline and GRPO reward-shaping design
- Hydra configuration hierarchy and DVC pipeline / remote storage design
- W&B / Weave and Hugging Face Hub integration points
- CI/CD workflow design, artifact/data flow, model promotion, testing, and deployment architecture

2. Reference Documents

Reference	Source	Applicability
TRD-ASKBEFOREANSWER-001	Internal	Functional & performance requirements this design satisfies
DVC Documentation	dvc.org	Pipeline stage & remote storage design

Reference	Source	Applicability
Hydra Documentation	hydra.cc	Configuration composition patterns
Weights & Biases Docs	docs.wandb.ai	Sweeps, registry, artifact lineage API
Unsloth Documentation	unsloth.ai/docs	FastLanguageModel loading, Triton kernels
Hugging Face Hub Docs	huggingface.co/docs	Model/dataset repo & Spaces deployment API

3. Repository / Package Architecture

The central design principle is that **large or generated ML artifacts do not become part of the Git repository**. Git tracks code and configuration; DVC tracks the data and model artifacts those code paths produce.

Path	Purpose
app/	Streamlit Hugging Face Space UI (app.py, requirements.txt)
configs/	Hydra YAML configuration hierarchy (model, data, training, infra)
data/	Processed dataset files — DVC-tracked, ignored by Git
docs/	Comprehensive lifecycle documentation (9-phase architecture)
models/	Model checkpoints — DVC-tracked, ignored by Git
scripts/	Executable CLI entry points (train_sft.py, evaluate.py, run_sweep_trial.py, ...)
src/ask_before_answer/data/	Preprocessing & synthetic data-generation logic
src/ask_before_answer/training/	Training orchestrators for SFT / DPO / ORPO / GRPO
src/ask_before_answer/evaluation/	Dual-scoring evaluation logic (LLM-judge + ActionScorer)
src/ask_before_answer/inference/	ClarifyOrActPipeline generation & parsing logic
sweeps/	W&B sweep orchestration configurations
tests/	Pytest unit tests
.github/workflows/	GitHub Actions CI and HF Space deployment workflows
dvc.yaml / .dvc/	DVC pipeline stage definitions and internal cache metadata
Makefile	Reproducible command aliases across the full lifecycle
.env.example	Environment secrets template (HF_TOKEN, GEMINI_API_KEY, WANDB_*)

4. Python Module Structure

The package under src/ask_before_answer/ is organized by lifecycle stage rather than by model type, so a single fine-tuning method can be added or removed without touching unrelated modules.

```
src/ask_before_answer/
  |-- data/         preprocessing.py, schema.py, facet_extraction.py
  |-- training/     trainer.py, sft.py, dpo.py, orpo.py, grpo.py, rewards.py
  |-- evaluation/   evaluate.py, action_scorer.py, judge_scorer.py
  |-- inference/    pipeline.py  (ClarifyOrActPipeline)
  `-- common/       config_schema.py, logging.py, io.py
```

4.1 Design Rules

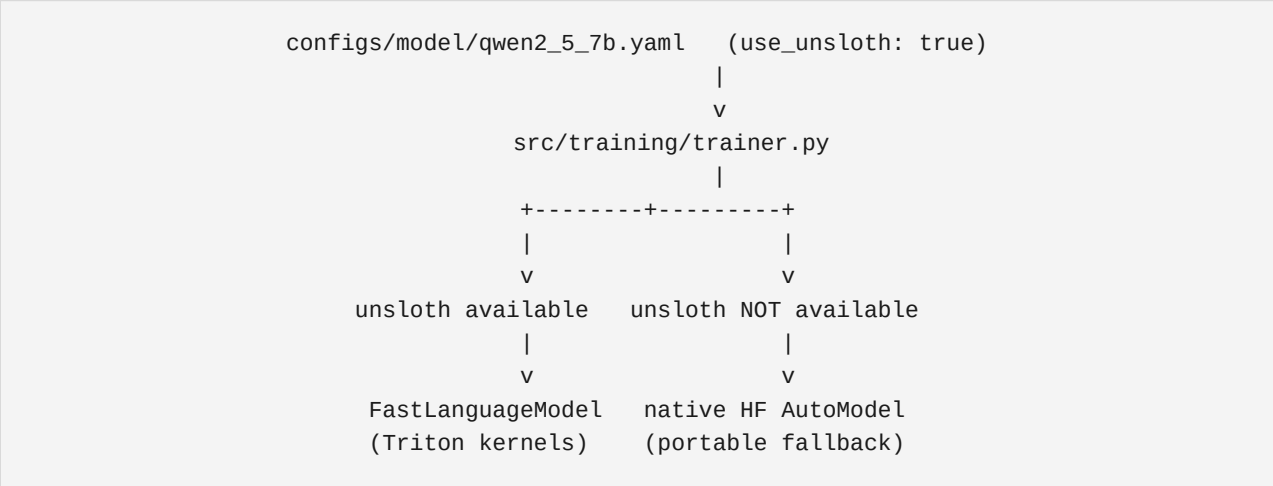
ID	Design Rule
MR-1	training/ modules depend only on data/ output contracts and common/, never on evaluation/ or inference/.
MR-2	inference/pipeline.py is the only module imported by app/app.py, keeping the deployment payload minimal (Section 15).
MR-3	Every fine-tuning method (sft.py, dpo.py, orpo.py, grpo.py) implements a shared TrainerBase interface so the Makefile dispatch layer (Section 11) is uniform across variants.
MR-4	Reward functions for GRPO live in a dedicated rewards.py module, decoupled from grpo.py, so reward_weights overrides in Hydra do not require code changes.

5. Qwen / Unsloth Model Architecture

The base model is **Qwen 2.5 7B Instruct**, fine-tuned with LoRA adapters. Model loading is abstracted so the same training entry point transparently uses either accelerated Unsloth loading or native Hugging Face loading, depending on environment capability.

5.1 Hardware Acceleration Stack

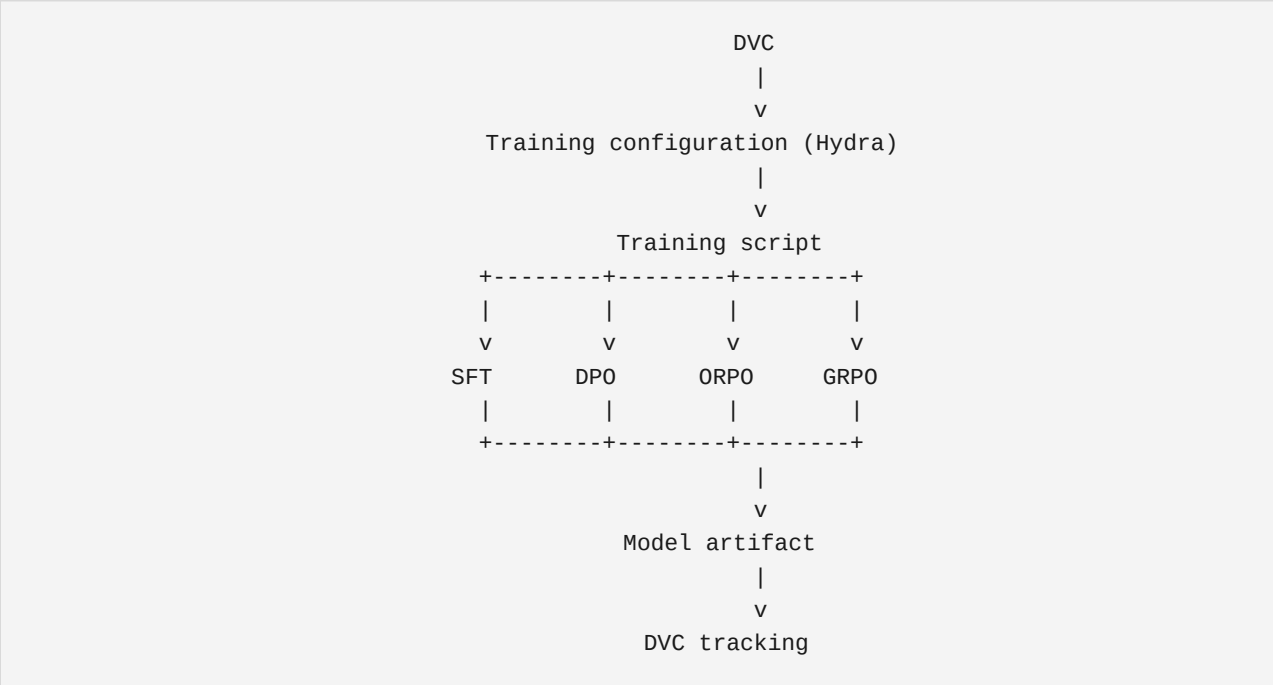
Layer	Role
Flash Attention (flash-attn)	Computes attention without materializing the full N×N matrix, reducing memory from O(N^2) to O(N). Targets Ampere+ GPUs (RTX A6000, 3090, 4090).
xFormers	Meta's optimized transformer building blocks; fallback attention path for older architectures (Turing, Volta).
Unsloth	Sits atop Flash Attention for core attention; supplies custom Triton kernels for LoRA weight updates (via bitsandbytes 4-bit/8-bit QLoRA), the Cross-Entropy loss, RoPE, and MLP blocks.



NOTE Unsloth compiles Triton kernels on the fly and strictly requires the CUDA toolkit on the environment path. On institutional HPC/Slurm clusters, the CUDA module (e.g. module load cuda/12.6) MUST be loaded before the training pipeline runs, or Unsloth initialization fails.

6. SFT → DPO / GRPO Training Pipeline

Training deliberately separates the notion of a **model** (the trained artifact), a **fine-tuning method** (SFT, DPO, ORPO, GRPO), and a **training variant** (a specific DVC configuration, e.g. sft-only or dpo-only).



6.1 Stage Dispatch

```
make train TRAIN_VARIANT=sft      # generic dispatch
make train-sft                    # readable alias
make train-dpo
make train-orpo
make train-grpo
```

6.2 GRPO Reward Shaping Design

GRPO relies entirely on programmatic reward functions rather than static contrastive pairs. Four decoupled reward functions, each independently weighted via configs/training/grpo.yaml reward_weights, jointly shape the policy:

Function	Design Intent
format_reward_func	Positive reward only if all four schema headers (Action:, Reasoning:, Facets:, Response:) are present
action_reward_func	Penalizes an incorrect Action choice against dataset ground truth
facet_logic_reward_func	Enforces Clarify → non-empty Facets, and Answer → empty Facets
accuracy_reward_func	Token F1 overlap between generated Response and the factual ground-truth answer; the primary anti-hallucination signal, and the term that resolves format-only reward hacking

7. Hydra Configuration Structure

All pipeline hyperparameters, data paths, preprocessing switches, evaluation thresholds, and infrastructure settings are managed through Hydra (v1.3+). No hardcoded values are permitted in pipeline code.

Config Path	Governs
conf/config.yaml	Root config — imports all sub-configs via the defaults list
conf/model/qwen2_5_7b.yaml	Base model identity, LoRA rank, use_unsloth toggle
conf/data/ambignq.yaml	AmbigNQ source paths, schema settings, split ratios
conf/training/sft.yaml	SFT learning rate, batch size, epochs, LoRA rank
conf/training/dpo.yaml	DPO beta, learning rate, chosen/rejected dataset paths
conf/training/grpo.yaml	GRPO beta, learning rate, reward_weights block (Section 6.2)
conf/infra/local.yaml / hpc.yaml	Runtime environment: local GPU vs. HPC/Slurm CUDA module settings

Runtime parameter overrides follow the standard Hydra CLI pattern, e.g. python scripts/train_dpo.py training.beta=0.1 training.learning_rate=5e-7, allowing sweep agents (Section 9) to override configuration without editing YAML.

8. DVC Pipeline & Remote Storage Design

DVC provides reproducible pipeline execution, data/model dependency tracking, and experiment management. Every pipeline stage is declared in dvc.yaml with explicit deps and outs, so dvc repro can recompute only the stages whose inputs changed.

```
                    [data/raw] --preprocess--> [data/processed]
                                                    |
                                                    v
                                [train: sft|dpo|orpo|grpo]
                                                    |
                                                    v
                                         [models/<variant>]
                                                    |
                                                    v
                                              [evaluate]
                                                    |
                                                    v
                                  [reports/metrics, reports/figures]
```

8.1 Experiments & Sweep Isolation

Because DVC tracks the exact YAML config state for every sweep trial, winning hyperparameters never need to be copy-pasted: dvc exp apply sweep_<Run ID> reverts local configuration files to that exact optimal state, ready for a Git commit as the new defaults.

NOTE Manual ablation runs (make ablation-suite, plain dvc repro) are not tagged with the W&B .sweep property. The ablation report generator skips any run carrying that tag, keeping the final baseline leaderboard free of sweep noise.

9. W&B / Weave Integration

Integration Point	Design
Training telemetry	Every DVC training stage streams loss, reward components, and eval metrics live to W&B via the Trainer callback interface
Bayesian sweeps	scripts/run_sweep_trial.py is the W&B sweep agent entry point; it fetches hyperparameters, patches the local Hydra config, and runs dvc exp run
Sweep reporting	scripts/generate_sweep_report.py pulls cloud metrics grouped by Sweep ID, plots Validation Curves, and statelessly regenerates docs/sweep_report.md
Model registry	publish-model-artifact creates/verifies an immutable W&B artifact and checks the production alias + registry digest before any Hugging Face push is permitted
Weave evaluation	scripts/evaluate.py runs the dual-scoring pipeline (LLM-as-a-judge + ActionScorer) through Weave, producing a dynamic leaderboard

Integration Point	Design
Weave tracing	app/app.py wraps every user interaction, prompt, and generation in a Weave trace for real-time production observability

9.1 Advanced Interactive Charts (W&B Dashboard)

- **Parallel Coordinates Chart** — traces how hyperparameter combinations flow toward final Eval Loss
- **Hyperparameter Importance Matrix** — a Random Forest trained on sweep results in real time to rank feature importance
- **Interactive Validation Curves** — scatter plots of performance vs. hyperparameter, explored live in the browser

10. Hugging Face Integration

Hugging Face Hub is the publication surface for the final dataset, the production model, and the public inference Space. Deployment to Hugging Face is only permitted from a verified W&B production artifact (Section 13).

Artifact	Hugging Face Repository Type	Publication Trigger
AskBeforeAnswer dataset	Dataset repo	make deploy-hf, after Data Card regeneration
AskBeforeAnswer model	Model repo	make deploy-hf, gated by W&B provenance verification
Inference demo	Space repo (Docker/Streamlit)	GitHub Release publish event (Section 14)

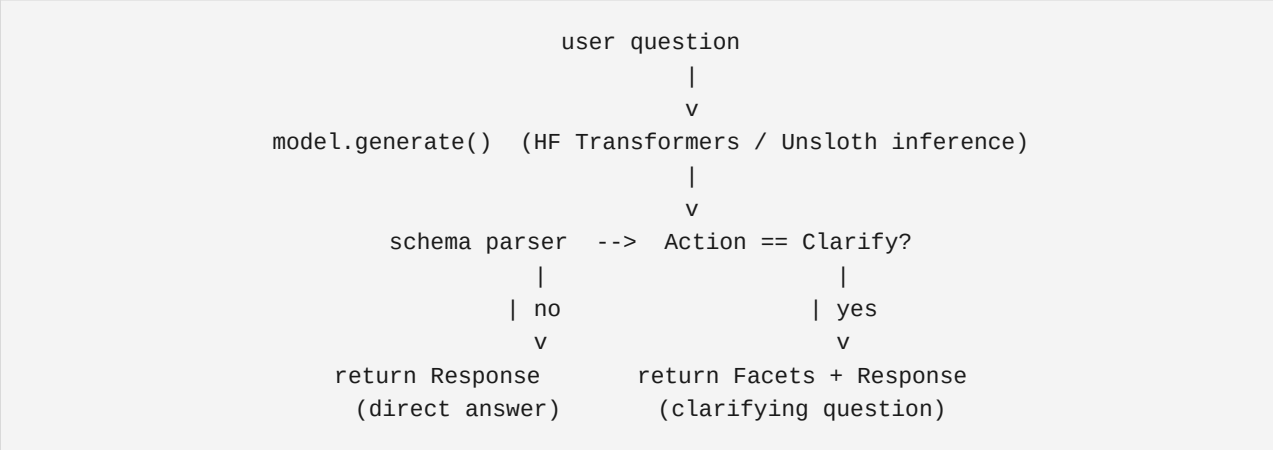
11. Makefile Command Reference

Command	Purpose
make install / make install-dvc	Install Python dependencies and DVC (via uv or pipx)
make preprocess	Run the 3-stage synthetic data generation pipeline (DVC-tracked)
make run-pipeline	Full DVC pipeline: data → SFT → SFT eval → DPO → SFT+DPO eval
make train TRAIN_VARIANT=<v> / make train-sft / -dpo / -orpo / -grpo	Run a specific training variant
make sweep FINE_TUNE_METHOD=<m> COUNT=<n> / make sweep-sft / -dpo / -grpo	Initialize and run a W&B Bayesian sweep
make ablation-suite	Run the clean baseline dvc repro and regenerate the ablation report

Command	Purpose
make evaluate	Run the dual-scoring evaluation suite and publish the Weave leaderboard
make infer	Interactive CLI inference
make run-app	Launch the local Streamlit UI
make promote-dvc MODEL=<m> EXPERIMENT=<id>	Select the winning DVC experiment as release source of truth
make publish-model-artifact MODEL=<m> EXPERIMENT=<id> STAGE=production	W&B release gate: validate, create/verify artifact, record provenance
make deploy-hf	Push the verified model/data artifact to Hugging Face
make lint / make test	CI-equivalent static checks and pytest suite

12. Inference Architecture

At the core of the inference framework is the **ClarifyOrActPipeline** (src/ask_before_answer/inference/pipeline.py). It parses a raw model generation into the 4-field schema and routes the response through a dual-action mechanism: an ambiguous query returns Action: Clarify with facets and a targeted question; a clear query returns Action: Answer and a direct response.



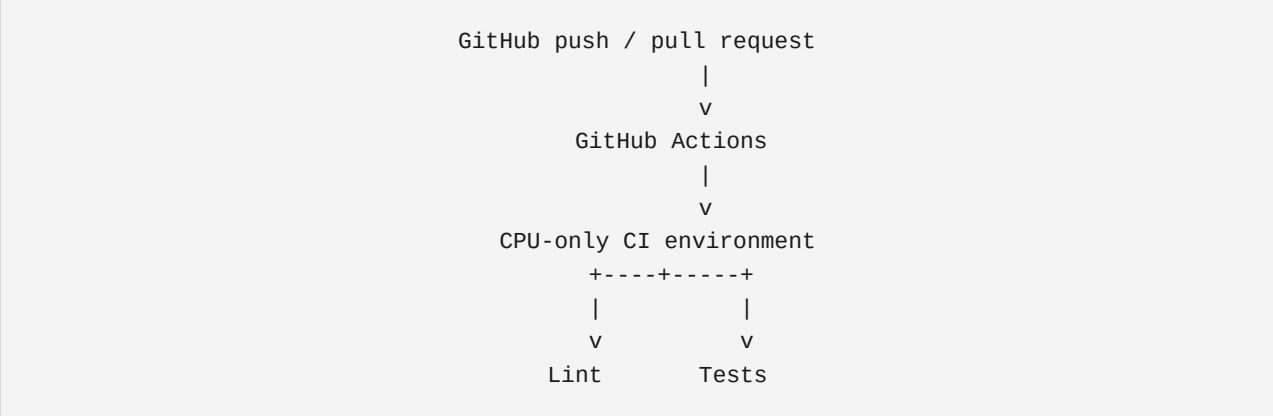
12.1 Serving Configuration

Tier	Hardware	Precision
Production GPU serving	1x NVIDIA T4 / L4 / RTX 3060 (8–12GB VRAM)	4-bit / 8-bit quantized (bitsandbytes)
CPU-only inference	Any x86/ARM host, no GPU required	bitsandbytes or llama.cpp quantization

NOTE The current production serving path is Hugging Face Transformers (optionally Unsloth-accelerated) behind the Streamlit application, not a dedicated high-throughput server. A vLLM (or TGI-class) serving backend is a reasonable future upgrade if concurrent request volume grows beyond what the single-process Streamlit deployment can sustain; it is tracked as an open item in the companion TRD rather than a current requirement.

13. CI/CD — GitHub Actions

Continuous Integration runs on every push and pull request to main, in a CPU-only environment. CI deliberately does not reproduce the GPU training environment, since training has substantially different dependencies (CUDA-enabled PyTorch, Unsloth, xFormers, Flash Attention).



13.1 Deployment Workflow

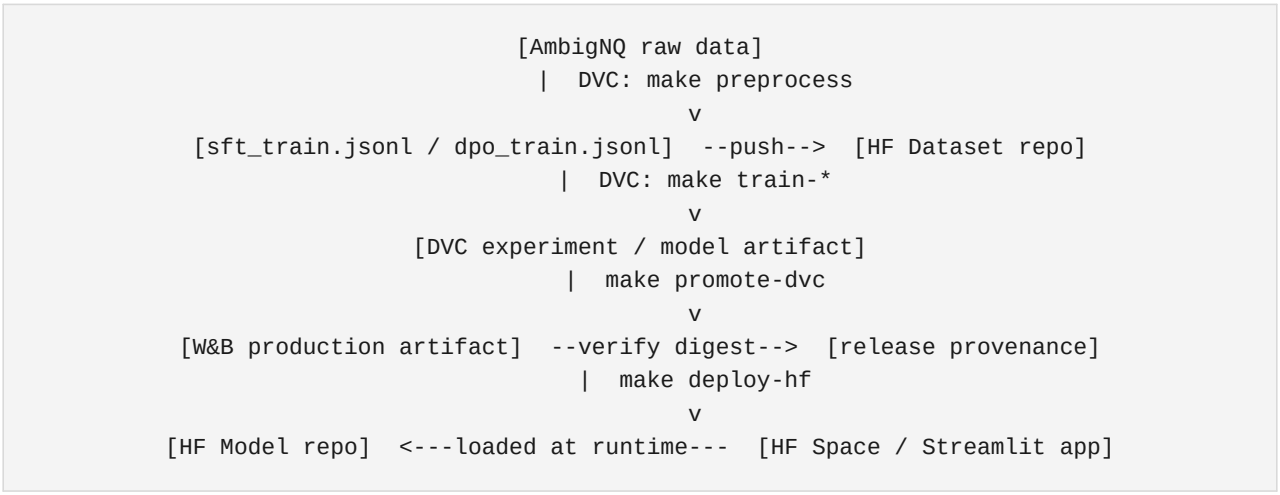
The HF Space demo is deployed through a separate workflow, triggered by a published GitHub Release rather than by the model publication command, so the model and the application can evolve independently.

```
# .github/workflows/deploy-hf-demo.yml
on:
  release:
    types: [published]
```

14. Data Flow & Artifact Flow

End-to-end, the system routes control and artifacts through five distinct systems, each with a deliberately limited responsibility, enforcing a strict chain of custody from raw data to a publicly served model.

System	Primary Responsibility
Git / GitHub	Source code, configuration, CI, releases
DVC	Versioned datasets, model artifacts, reproducible experiments
W&B	Training telemetry, hyperparameter sweeps, evaluation results, model registry
Hugging Face Hub	Public model and dataset publication
GitHub Actions	Continuous integration and application deployment
Hugging Face Space	Public Streamlit inference application



15. Model Promotion & Provenance

The deployment pipeline is deliberately split into two stages to preserve chain of custody and enforce a human-in-the-loop review: automation executes sweeps, generates telemetry, and securely deploys finalized assets, but never automatically selects the promotion “winner.”

Stage	Command	Output
DVC promotion	make promote-dvc MODEL=<model> EXPERIMENT=<id>	Selected DVC experiment marked as release source of truth
W&B release gate	make publish-model-artifact ... STAGE=production	Immutable W&B artifact + verified registry digest + production alias + local provenance record
Hugging Face deployment	make deploy-hf	Verified artifact pushed to HF, Model/Data Card updated with release notes and metrics

IMPORTANT Hugging Face deployment is permitted only from a W&B production artifact whose exact identity and immutable digest have been recorded in release provenance and independently verified at deployment time. promote-dvc alone never deploys a model.

16. Testing Architecture

Layer	Scope	Tooling
Unit tests	Schema validation, facet parsing, reward function logic, config composition	pytest, tests/
Static checks	Lint, formatting, type checks	GitHub Actions CI (CPU-only)

Layer	Scope	Tooling
Pipeline smoke tests	make preprocess / make evaluate against a small fixture split	DVC + pytest fixtures
Evaluation regression	Six-variant leaderboard compared against Section 6 thresholds of the TRD	W&B Weave

CI must pass make lint and make test before merge; GPU-dependent training and inference code paths are exercised outside CI, in the DVC-orchestrated training stage, since they require CUDA hardware not present in the CI runner.

17. Deployment Architecture

The demo application's payload is intentionally minimal: app/ plus the single inference module src/ask_before_answer/inference/pipeline.py. The data, training, and evaluation modules are excluded, keeping the deployed image lightweight.

```
app/
|- app.py
\- requirements.txt
+ src/ask_before_answer/inference/pipeline.py
```

Property	Detail
Application	Streamlit inference UI, Docker-packaged on Hugging Face Spaces
Deployment trigger	Published GitHub Release (Section 13.1), not model publication
Model source at runtime	Hugging Face model repository (never a bundled checkpoint)
Observability	All prompts and generations traced to W&B Weave in real time
Independent evolution	Model and application version independently; a new model release does not require redeploying the app, and vice versa

18. Revision History

Version	Date	Author	Change Summary
1.0.0	Sep 2026	ML Engineering & MLOps	Initial draft for review

END OF DOCUMENT | TDD-ASKBEFOREANSWER-001 v1.0.0 | INTERNAL / ENGINEERING